

Home

Dashboard

My own courses

Schools

Course feedback

Service Links

Intelliboard

🔍

🔔

📧

👤

☰

CS-E4960 - Software Testing and Quality Assurance D, Lecture, 3.9.2024-26.11.2024

This course space end date is set to 26.11.2024 **Search Courses: CS-E4960**

?

Assignments

Attendances

External tools

Forums

Group choices

Questionnaires

Quizzes

Resources

Syllabus

Course feedback

/ Department of Computer Science / Sections / Individual assignments / Unit Testing 3

Unit Testing 3

To do: Make a submission

To do: Receive a grade

Opened: Tuesday, 1 October 2024, 10:00 AM

Due: Tuesday, 8 October 2024, 10:00 AM

Overview

Test automation can help make the execution of tests more efficient and consistent, but how about test design? Could it be automated, too?

In this assignment, your task is to explore the automatic generation of unit test cases by [large language models](#) (LLMs). An LLM is an artificial neural network that takes an input text and repeatedly predicts the next word or token. LLMs have been shown to be able to produce code from a human-readable description (a "prompt"). In our case, the prompt consists of a textual instruction that asks the LLM tool to produce unit test cases and the code for which to produce the test cases. The goal of this assignment is that you explore how you can get the LLM tool to produce test cases for Python's unittest module and to analyze the merits of the test cases.

In this assignment, you must use a tool such as [ChatGPT](#), [Mistral](#), [Gemini](#), [GitHub Copilot](#), or any other LLM-based AI tool of your choice.

To complete this assignment, do the following:

1. Download the attached zip file with code. Note: This is the same code that you used for the Unit Testing 1 assignment!

2. Write a prompt for your AI tool to create unit tests for the following methods. The general idea of the test is given below:

1. `set_difficulty` – Validate that the difficulty is correctly set and that invalid difficulty levels raise a `ValueError`.
Note: This method is not implemented! You should ask the AI tool to create the test first without the implemented method. When you run the resulting unit test, the test suite should fail. Then, implement the `set_difficulty` method and rerun the test suite. The test should now pass. (See below for a precise specification of the function. You are allowed to use your implementation from previous assignments.)

2. `place_bet` – Verify that valid bets within the allowed range are accepted and that invalid bets raise a `ValueError`.

3. `guess_number` – Ensure that guesses are correctly evaluated (higher, lower, correct) and that guesses outside the range (1-100) raise a `ValueError`.

4. `double_prize` – Verify that doubling the prize works correctly based on the player's choice, including the outcome being random.

5. `save_bet` – Verify that saving the bet works correctly based on the player's choice, including the outcome being random.

6. `add_score` – Test that scores are added correctly and that invalid scores (e.g., negative values) raise a `ValueError`.

7. `save_scores` – Test that scores are saved correctly and that invalid scores (e.g., negative values) raise a `ValueError`.

8. `get_scores` – Verify that the correct top 10 scores are returned.

9. `load_scores` – Test that scores are loaded correctly from the file and that invalid lines are skipped.

3. Run the prompt on the AI tool.

4. Analyze the result you get from the AI tool in relation to the guidelines introduced in the course (see below). For each guideline, write down your analysis of how well the AI-generated test cases match them. Remember to write the "why" – the reason why you think that a guideline is matched or not. (E.g., "Well named: not matched because <reason>".)

5. Based on your observations, modify your prompt and go back to step 3. Write down how the result changed. If you think the result is perfect, you can go to the next step. Do a maximum of three iterations including the first one.

6. Write a summary of the process of trying to design tests automatically. Points to consider:

1. What kind of test design did the AI use?

2. Did it miss some crucial point?

3. What did you do differently when you did the Unit Testing 1 assignment? Why?

4. Would the AI-generated tests work if the SUT was modified?

5. Would the AI-generated tests work if a new fault was introduced in a future version of the SUT? Try it: change code to introduce a fault and see if the tests catch it.

7. End the summary by answering this question: Based on your exploration, to what extent can LLM-based AI be used to speed up test case design while ensuring test correctness? Motivate your answer by using the findings of your exploration. (In other words, give examples from what you found to support your answer. Remember to clearly mark what parts are AI-generated to distinguish them from parts you have written yourself.)

8. Put your notes and answers into a file and submit it as PDF. See below for a template structure that makes it easy to see how your exploration has proceeded.

Specification for `set_difficulty`

The function signature for set difficulty is:

```
def set_difficulty(self, difficulty):
```

The `set_difficulty` method sets the difficulty level of the game, which determines the number of maximum guesses allowed.

Parameters:

`difficulty` (str): A string representing the desired difficulty level. Acceptable values are 'easy' and 'hard'.

Behavior:

If difficulty is 'easy', the game allows a maximum of 10 guesses. (i.e., `self.max_guesses`)

If difficulty is 'hard', the game allows a maximum of 5 guesses.

If the provided difficulty is not 'easy' or 'hard', the function raises a `ValueError` with a message indicating that the difficulty must be 'easy' or 'hard'.

Some important guidelines for unit tests

- **Well named:** The test name explains what it tests.
- **Clearly defined:**
 - Each test tests one thing.
 - Inputs and outputs are unambiguous.
- **Repeatable (deterministic):** The test gives the same results no matter how many times it is run.
- **Isolated:**
 - No interactions or dependencies between tests.
 - No dependencies on outside factors (e.g., file system, database).
 - Each test leaves the system in a well-defined state.
- **Robust:** The test does not break if the unit internals are changed (related to isolation).
- **Fast:** The total testing time is reasonable even with thousands of tests.
- **Documented:** Explain the purpose of the test, the test scenario, and the expected behavior when the scenario is invoked.

Assignment template

Unit testing 3

<name>

<date>

Environment description:

<short description of the AI tool you used>

Prompt 1:

<the prompt that you created in step 2>

Result 1:

<the result from the AI>

Analysis 1:

<a point-by-point analysis against the guidelines>

Prompt 2:

<explain what changes you made to the prompt and why>

<the new prompt>

Result 2:

<the result from the AI>

Analysis 2:

<a short analysis of how the result changed and how it corresponds to the guidelines>

Prompt 3:

<explain what changes you made to the prompt and why>

<the new prompt>

Result 3:

<the result from the AI>

Analysis 3:

<a short analysis of how the result changed and how it corresponds to the guidelines>

Summary:

<a summary of your exploration, incorporating the points mentioned in step 6>

<your answer to the final question in step 7>

Submission: You must submit your answer as a single PDF file named as studentnumber-UT3.pdf replacing "studentnumber" with your actual student number.

Grading:

- 3-2 points: The answer as a whole is clear and insightful: the prompts are sensibly chosen; the analysis displays critical thinking; and the summary raises important points and gives a reasoned answer to the final question.
- 2-1 points: The answer as a whole is not completely clear or includes few insights: the prompts work but could have been more aligned with the desired goals; the analysis misses some critical points; or the summary misses important points or lacks clear reasoning in the final question.
- 1-0 points: The answer as a whole lacks insights: the prompts have only a weak connection to the desired goals; the analysis misses most or all critical points; or the summary lacks critical thinking and the final question is answered without reasoning.
- 0 points: The answer lacks most parts; is not understandable, or the reasoning is severely lacking; the answer includes only the prompts and results without reasoning or explanation.

Use of AI and Collaboration with Other Students: Using AI tools to produce an answer for this assignment is permitted. However, you must clearly indicate which parts of the assignment are produced by AI and which parts you wrote yourself. Collaboration with other students is not permitted. This is an individual assignment that you must author independently.

UnitTesting3.zip

10 September 2024, 9:26 AM

Submission status

Attempt number	This is attempt 1.
Submission status	No submissions have been made yet
Grading status	Released
Time remaining	Assignment is overdue by: 155 days 4 hours
Last modified	-

Previous activity

➔ Unit Testing 2

Next activity

Acceptance Testing 1: rf-katas ➔

A!

Aalto-yliopisto

Aalto-universitetet

Aalto University

Students

- MyCourses instructions for students
- Support form for students

Teachers

- MyCourses help
- MyTeaching Support

About service

- MyCourses protection of privacy
- Privacy notice
- Service description
- Accessibility summary

MyCourses support for students

Nguyen Binh (Log out)